

1. Implement unit tests, integration tests, and end-to-end tests for a sample application using frameworks like JUnit, TestNG, or Selenium?

- JUnit – for unit and integration testing (Java)
- TestNG – alternative to JUnit (not required but optional)
- Selenium – for E2E (UI) testing in the browser

We'll walk through this using a simple Java Spring Boot web app example.

Sample Application (Spring Boot)

Endpoint Example:

```
java
```

```
// HelloController.java
```

```
@RestController
```

```
public class HelloController {
```

```
    @GetMapping("/hello")
```

```
    public String sayHello() {
```

```
        return "Hello World";
```

```
    }
```

```
}
```

1. Unit Tests with JUnit

Unit tests test individual methods without any external dependencies.

HelloControllerTest.java

```
java

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class HelloControllerTest {

    @Test
    void testSayHello() {
        HelloController controller = new HelloController();
        String response = controller.sayHello();
        assertEquals("Hello World", response);
    }
}
```

Run using: mvn test or your IDE

2. Integration Tests with Spring Boot + JUnit

Integration tests test the interaction between components, like REST controllers and services.

HelloIntegrationTest.java

java

```
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.junit.jupiter.api.Test;
import static org.assertj.core.api.Assertions.assertThat;
```

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
```

```
public class HelloIntegrationTest {
```

```
    @Autowired
```

```
    private TestRestTemplate restTemplate;
```

```
    @Test
```

```
    void testHelloEndpoint() {
```

```
        String body = this.restTemplate.getForObject("/hello", String.class);
```

```
        assertThat(body).isEqualTo("Hello World");
```

```
    }
```

```
}
```

This starts the full Spring Boot app on a random port for testing.

3. End-to-End (E2E) Tests with Selenium

Selenium simulates a real browser and tests the UI + backend together.

Maven Dependencies

Add to your pom.xml:

xml

```
<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.19.0</version>
  </dependency>
</dependencies>
```

HelloSeleniumTest.java

java

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import static org.junit.jupiter.api.Assertions.*;
```

```
public class HelloSeleniumTest {
```

```
    private WebDriver driver;
```

```
    @BeforeEach
```

```
void setup() {  
    // Make sure chromedriver is in your PATH  
    driver = new ChromeDriver();  
}  
  
@Test  
void testHelloPage() {  
    driver.get("http://localhost:8080/hello");  
    String pageSource = driver.getPageSource();  
    assertTrue(pageSource.contains("Hello World"));  
}  
  
@AfterEach  
void teardown() {  
    if (driver != null) {  
        driver.quit();  
    }  
}  
}
```

Summary of Tests

Test Type	Tool	Scope	Example
Unit Test	JUnit	Single class/method	HelloControllerTest
Integration	JUnit + Spring	Multiple components (REST, services)	HelloIntegrationTest
End-to-End	Selenium	UI + Backend	HelloSeleniumTest